MACHECK CODEBASE REFACTORING WALKTHROUGH

This document outlines the refactoring changes made to the `mobile-MaCheck` codebase. All screens and hooks have been modernized to use a unified Zustand store, eliminating redundant local states and manual `AsyncStorage` operations.

---

1. Unified State Management (`Zustand` Store)

We created a central Zustand store at useAppStore.ts to manage:

- **Active User Profile**: Synced automatically to backend and local AsyncStorage.
- **All Registered Profiles**: Quick user switcher list.
- **Medicine Cabinet**: Offline local database with cloud sync.
- **Activity Logs**: Audit trail of patient events.
- **Active Challenge State**: Spacing timer and water cup records.
- **Global Preferences**: Font size, doctor mode, sound muted, and server URL configs.

---

2. Refactored Screens

Every application screen was updated to consume Zustand store selectors and actions, removing duplicate local states:

1. **Home Screen** (index.tsx): Powered by Zustand selectors. Integrated with `<CustomAlertModal>` for alerts. Spacing challenge widget dynamically binds to `activeChallenge`.
2. **Cabinet Screen** (cabinet.tsx): Converted from manual AsyncStorage loads to Zustand select maps. Replaced custom medicine items with shared `<MedCard>` components.

3. **Register Screen** (register.tsx): Synchronizes user creation, settings adjustment, and quick user switching through Zustand store actions.
4. **Scanner Screen** (scanner.tsx): Pulls cabinet list directly from Zustand to execute offline drug interaction analyses instantaneously.
5. **Chatbot Screen** (chatbot.tsx): Consumes Zustand context without polling lag.
6. **Food Clash Screen** (food-clash.tsx): Performs safe foods & herbs checks against store profiles.
7. **Caregiver Screen** (caregiver.tsx): Observes logs and cabinet states instantly.

---

3. Shared Modular Hooks & Components

- **Shared Hooks**: use-sound.ts, use-font-size.ts, and use-doctor-mode.ts read directly from Zustand, guaranteeing instantaneous global updates without UI reloading.
- **Shared Components**: `<CustomAlertModal>` and `<MedCard>` unify aesthetics, support doctor mode (grayscale rendering), and adapt font sizes correctly.

---

4. Verification & Clean Compilation

We ran `npx tsc --noEmit` to verify type safety:

- Resolved duplicate `ActivityLog` interface clash.
- Corrected prop signatures in `<CustomAlertModal>` and `<MedCard>`.
- Aligned types for `UserProfile` and `CabinetMed` between store states and screen forms.
- TypeScript check now compiles successfully with **zero errors and warnings**!

---

5. Xcode & Native Build Improvements

We automated Metro bundler and backend server execution during iOS development:

- **Xcode Pre-Actions Script**: Updated target pre-action shell script in mobileMaCheck.xcscheme. The script checks if port 8081 (Metro) and port 5001 (Backend Server) are open. If not, it uses AppleScript (`osascript`) to open new macOS Terminal windows and run `npx expo start --port 8081` and `node server.js` automatically.
- **Native Tracking**: Un-ignored `/ios` and `/android` directories in .gitignore to ensure scheme improvements, package scripts, and native configs are checked in and shared with team members.

---

6. Git & Private GitHub Integration

We established a reliable version control and collaboration pipeline:

- **Local Repository Initialization**: Created a Git repository in `mobile-MaCheck` and configured local Git identities (`user.name "JameMy0001"` , `user.email "JameMy0001@users.noreply.github.com"`).
- **Initial Commit**: Committed 115 project and native configuration files.
- **Private Repository Integration**: Configured remote origin mapping to `https://github.com/JameMy0001/mobile-MaCheck.git` .
- **CLI Authentication and Upload**: Authenticated using GitHub CLI (`gh auth login`) via web browser and pushed all changes to the remote `main` branch.

---

7. Web-Based Doctor Dashboard (Doctor Portal)

We implemented a premium, medical-themed web portal served directly by the backend Express server:

- **Static Assets Hosting**: Added `public/` directory hosting at `app.use('/doctor', express.static(...))` in server.js.
- **Patients API**: Created `GET /api/doctor/patients` to return structured profiles from Supabase.
- **Gemini AI Clinical Evaluation API**: Created `POST /api/doctor/summary/:phone` to invoke Gemini 1.5 Flash. It compiles patient age, diseases, allergies, current cabinet meds, and past logs into a comprehensive clinical summary markdown.

- **Responsive Doctor Portal UI**:
  - **Sidebar Navigation**: Dynamic list of registered patient profiles with real-time text searching and nudge indicators.
  - **Patient Detail Card**: Shows patient demographics, calculated ages, and active caregiver contact info.
  - **Allergies & Diseases Badges**: Emphasizes clinical warning alerts (Severe / Moderate) using color-coded tags.
  - **Interactive 2D Matrix Grid**: Visualizes active drug-drug clashes (Red for critical clashes, Yellow for caution, Green for safe).
  - **Timeline Audit Trail**: Pulls timeline logs, highlighting emergency actions or medication alerts.
  - **Cessation Warnings**: Parses patient logs to display stopped medications with detail logs (reasoning, commander, and time).
  - **AI Assessment Panel**: Seamlessly calls Gemini to render a detailed medical report using `marked` parser.
- **App Integration Button**: Added a dedicated **"เปิดแดชบอร์ดแพทย์ (Doctor Portal)"** button in the profile and settings tab of register.tsx. When tapped, it uses React Native's `Linking` API to open the local doctor dashboard url `http://localhost:5001/doctor` directly in the host device's web browser.